University of Sheffield

COM3529 Software Testing and Analysis

Larger Tests

Dr José Miguel Rojas

Larger Tests

Even with mocking, unit testing cannot serve all of our needs.

To check the behaviour of our code with that of others – or external services our code needs to work with – **integration tests** are needed.

To test the software as a whole, however, we need **system tests**.

There are a host of other reasons why we might want to write larger tests.

Why Write Larger Tests?

1. Fidelity
2. Unfaithful Doubles
3. Configuration Issues
4. Issues that arise under load
5. Unanticipated behaviours, inputs and side effects
6. Emergent Behaviours

Fidelity – Realism of the Environment

Fidelity is the property by which a test is reflective of the real behaviour of the system under test.

One way of envisioning fidelity is in terms of the **environment**:

- **Unit tests bundle a test and a small portion of the code together as a runnable unit, which ensures the code is tested, but is very different from how the production code runs.**

- Production is the environment of highest fidelity, but testing **should not** be carried out there! The best alternative is a **staged version of the system**, i.e., the system in its actual environment where testing can take place without using (or losing) real data.

Fidelity – Realism of Test Data

Tests can also be measured in terms of how faithful the test content is to reality.

Many handcrafted tests are dismissed by engineers if the test data itself looks unrealistic.

Test data copied from production is more faithful to reality, having being captured that way.

A big challenge is how to create realistic test traffic before launching new code!

Unfaithful Doubles

Doubles do not exactly replicate the thing they are doubling.

This can lead to gaps in unit tests.

Furthermore, doubles and their originals can become out of sync, particularly when the author of the original class is not also in charge of the double.

Configuration Issues

Unit tests cannot test configuration issues related to the whole system.

At Google, configuration changes are the number one reason for many major outages the company has experienced.

> **In 2013 there was a global Google outage because a bad network configuration was pushed that was never tested.**
>
> https://www.wired.co.uk/article/googledip

Issues that Only Arise Under Load

Unit tests cannot check for system performance and particularly its performance under extreme loads (this is called **stress testing**).

Large volumes are big! Often thousands or millions of queries per second.

Unanticipated behaviours, inputs and side effects

Unit tests are limited by the imagination of the engineer who wrote them!

Issues that users find with a product are mostly unanticipated (else they would have been tested for!)

This is where **automated testing *techniques*** (and AI) become useful – for example, **fuzzing**.

Emergent Behaviours

Unit tests are limited to the scope that they cover, so if behaviour changes outside of this scope it cannot be detected.

… and because unit tests are designed to be fast and reliable, they deliberately eliminate the chaos of real dependencies, network and data.

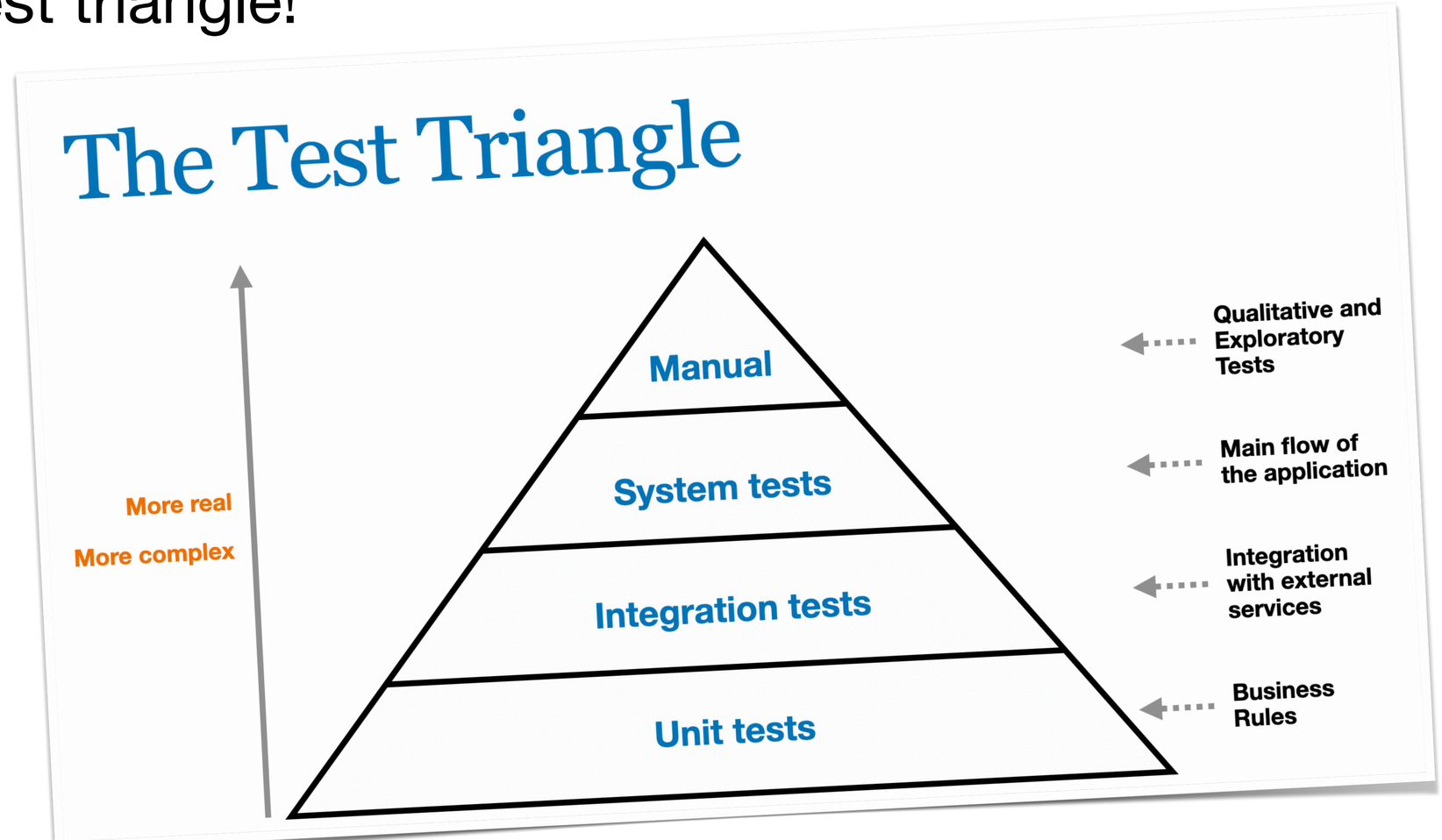
Unit tests are like problems in theoretical physics … ensconced in a vacuum, neatly hidden from the mess of the real world.

This is great for speed and reliability – **but can miss certain types of defects.**

These reasons are *not* to say “don’t do unit testing”

But rather that **larger tests should be used to complement unit tests.**

Recall the test triangle!



Some Types of System Tests

Browser and Device Testing

Browsers and devices vary, and often dictate how users interact with the application.

For example, what/how much content can be rendered, given the screen size.

Getting hold of all of these versions of devices can be problematic – leading to the setting up of public device libraries.

Performance, Load and Stress Testing

These test are critical for **ensuring that there is no degradation in performance between versions and that the system can handle expected spikes in traffic**.

Deployment Configuration Testing

Often it is not the code that is the source of defects, but the configuration:

- Data files
- Databases
- Option definitions

Larger tests can test these things because the configuration files are read during the launch of the product's binary.

Often these tests are **smoke tests** of the system, without needing much by way of additional test data or verification via assertions: **If the system starts successfully, the test passes!**

Exploratory Testing

Exploratory testing is a form of manual testing that focuses on looking for questionable behaviours by trying out new and established user scenarios.

Trained users/testers interact with a product, looking for new paths through the system for which behaviour deviates from expected or intuitive behaviour – or if there are security vulnerabilities.

User Acceptance Testing

Unit tests are written by developers, making it quite likely that misunderstandings about the intended behaviour of the system are reflected not only in the code but also in the unit tests.

User Acceptance Tests exercise the system to ensure the overall behaviour for specific user journeys is as intended.

Such tests are said to run the system from “end to end” and are therefore often called end-to-end (E2E) tests.

Multiple frameworks exist, such as Cucumber and RSpec, to make E2E tests writeable and readable in a user-friendly language.

Disaster Recovery

These tests expose how well the system reacts to unexpected changes or failures such as datacentre fires, malicious attacks, earthquakes etc.

Systems are taken out or taken down to observe the effects of what happens in practice.

This often tests the fragility of the organisation, too – as key decision makers may suddenly be out of contact.

Chaos Engineering

Made popular by Netflix, chaos engineering tests the resilience of systems by writing programs that simulate turbulent conditions.

These programs introduce faults/problems that include server shutdowns, latency injections, and resource exhaustion.

These faults are higher level and more targeted than those introduced by *mutation analysis*, which we will cover later in the course.

Problems with Larger Tests

Brittleness in Larger Tests

Larger tests do not escape from the problem of brittleness.

Recall that brittleness is when tests fail due to internal changes in code (refactoring) that do not affect its external behaviour.

Example:

> In end-to-end system testing, the tests could be too tightly coupled to the names of GUI elements or their layout – meaning that test fails when the UI changes.

Flaky Tests

Larger tests are more prone to suffer from flakiness.

Flakiness is a result of non-determinism in tests or code that cause tests to fail, when actually they work (and less common: vice versa).

Example: A system test that executes some code that requires a response from a server. When the server is running, the test passes, but when the server is down, the test fails. However, when the test fails, there's no fault in the code.

Unit tests can also be flaky, but since they tend to focus on logic and not involve non-deterministic elements, this is less common.

Problems Caused by Flaky Tests

The intermittency of flaky test failures is maddening for developers because the source of flakiness is **often hard to track down**:

> The symptoms of the problem (a failing test) are often far removed from the source of the non-deterministic behaviour, particularly with system tests.

But as soon as a test suite has several flaky tests, **developers tend to stop trusting it and/or running it**.

Flaky tests are a **frequent factor** in failing builds on continuous integration servers.

Problems with Larger Tests

1. **Reliability** – larger tests can be *brittle* and *flaky*
2. **Speed** – larger tests can be slow, interrupting a developer's workflow
3. **Ownership** – unit tests have an owner, who is in charge of system tests?
4. **Standardisation**. Unit tests have a standard frameworks and form. Larger tests have to cope with different infrastructures and environments, with various competing tools and frameworks (some of which are developed in house).